Group 23 - Digital Wallet

A desktop application that can securely store bank information, passwords, and authenticators in an all-in-one location.

Team:

- Joshua Pearson
- Isaac Miller
- Sam Prestigiacomo
- Alexander Toles
- Courtney McCray
- Jude Weller
- Jaiden Watts
- Bryant Goseland

Description:

Digital Wallet uses a layered architecture with four layers, presentation, feature, security, and storage. Dependencies point downward only. Feature modules never read files or call encryption routines directly. They ask the security and storage layers to do it, so mistakes in one feature cannot weaken the protection of another.

The Menu Controller is the presentation layer. It asks for the master password with hidden input, shows the main screen after unlock, and sends each menu choice to the right feature. The feature layer holds four modules, the Password Manager, the Authenticator, the Bank Card Wallet, and the Backup Service. The security layer holds the Vault Manager, the Key Manager, and the Crypto Service. The Vault Manager handles creating, unlocking, and locking the vault, including automatic locking after inactivity. The Key Manager derives one master key from the master password and then a separate key for each data file. The Crypto Service is the only component that performs key derivation, authenticated encryption, random number generation, and memory wiping. The storage layer holds the Secure File Store, which is the only component that touches the disk.

Each data set lives in its own encrypted file, passwords.enc, authenticator.enc, cards.enc, and backup.enc. A small vault.meta file holds the salt, key derivation settings, and format version. It contains nothing secret. Separate files and separate keys mean a fault in one file does not expose or corrupt the others, and each module loads only the data it needs.

The Menu Controller depends on the four feature classes and the Vault Manager. Each feature class owns a collection of its own records, the Password Manager holds Password Entries, the Authenticator holds Authenticator Accounts, and the Card Wallet holds Bank Cards. These are

composition relationships, so records exist only inside their manager. Every feature class depends on the Key Manager for its file key and on the Secure File Store for reading and writing. Only the Key Manager depends on the Crypto Service. This narrow dependency keeps the security-critical code in a few small classes that the team can review closely. Secret values are held as byte buffers instead of plain strings so they can be wiped when no longer needed. Bank Cards expose a masked number by default and reveal the full number only on request.

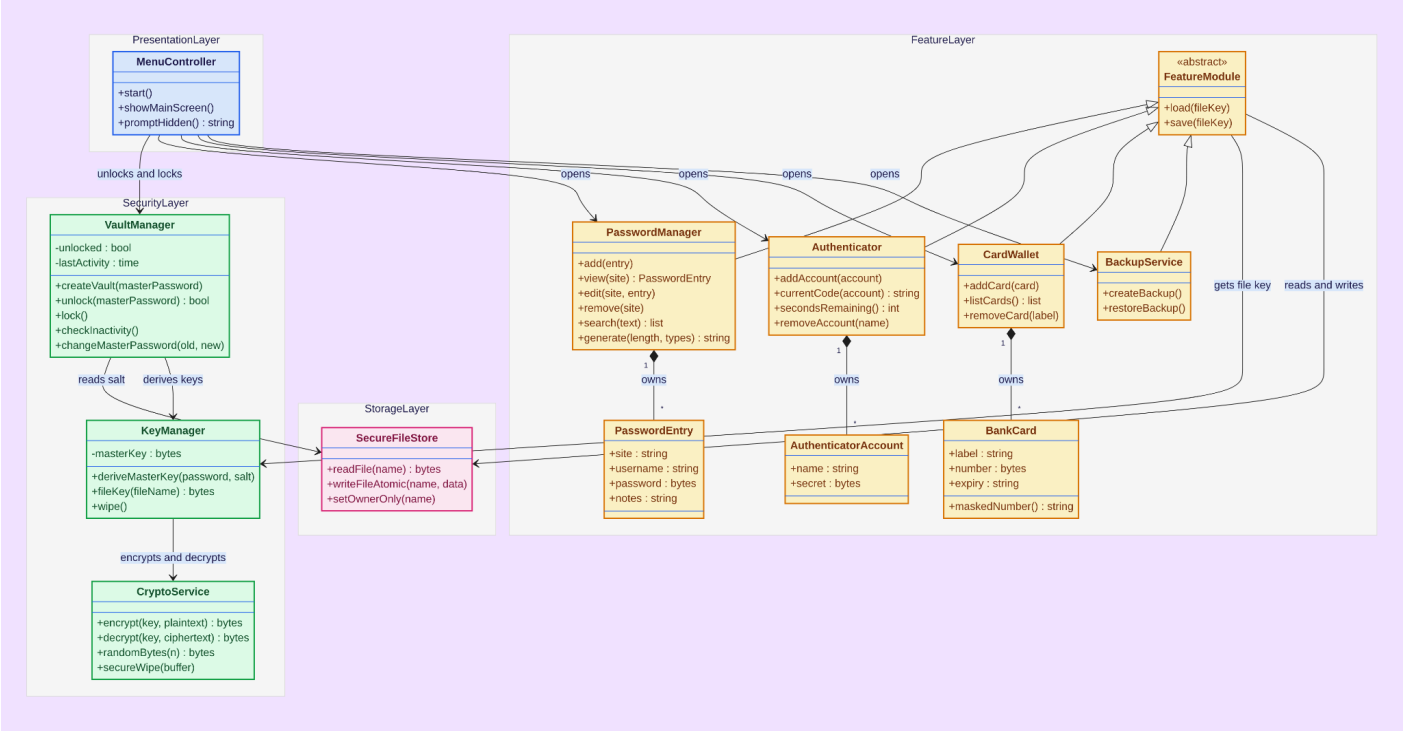
The user starts the executable and enters the master password. The Vault Manager reads the salt from vault.meta, and the Key Manager derives the master key using a slow, salted method. The Vault Manager then reads an encrypted file and asks the Crypto Service to decrypt it. If the password is wrong or the file was changed, authenticated decryption fails, and the user sees one generic error with no details. On success the main screen appears. When the user adds an entry, the Password Manager encrypts the updated data with a fresh nonce and asks the Secure File Store to save it atomically with owner-only permissions, so a crash cannot corrupt the vault.

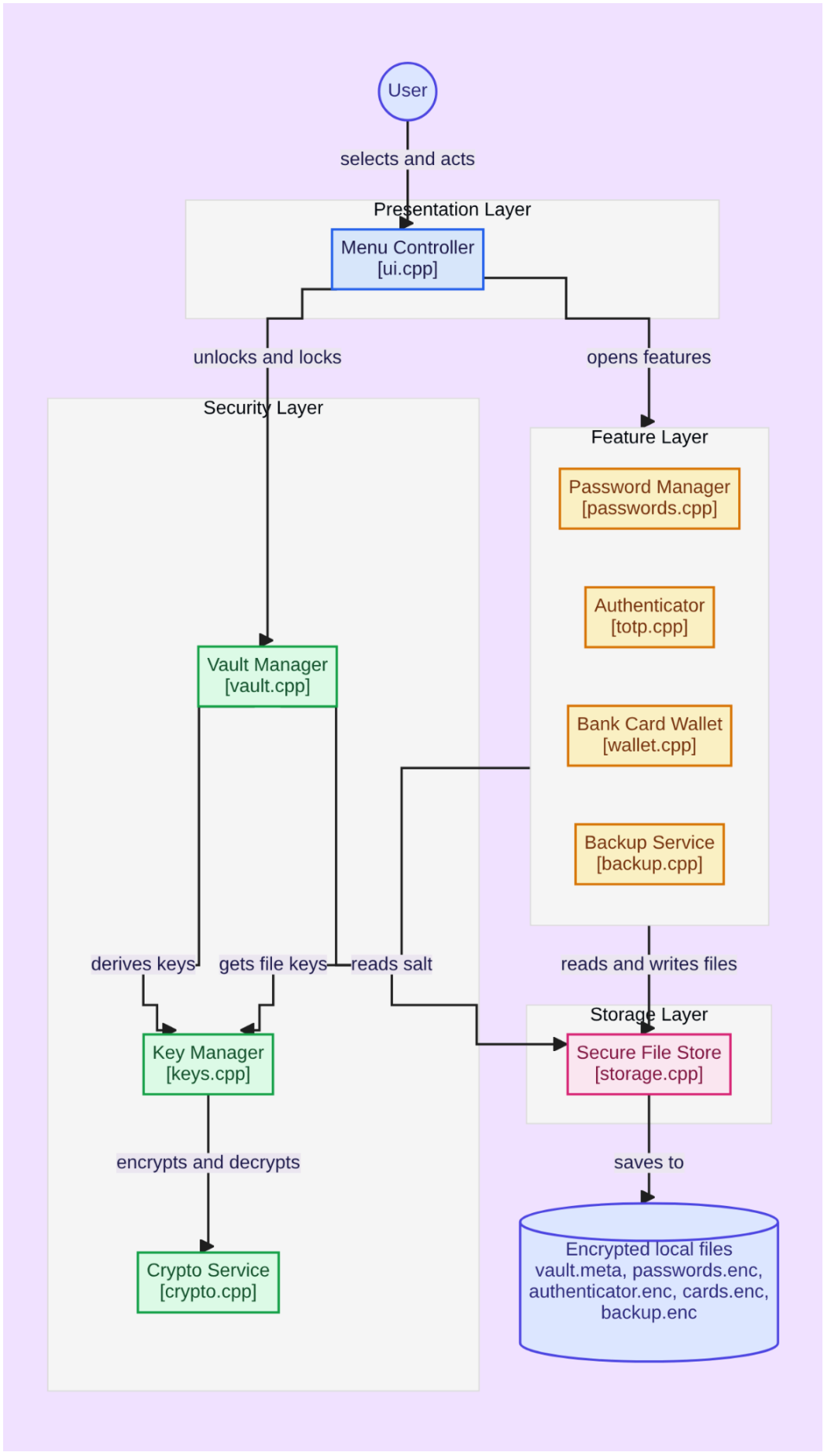
The master password is never stored. Keys are derived from it each session and wiped when the vault locks. Authenticated encryption detects tampering and corruption. Files are written to a temporary location and renamed, and they are restricted to the owner. Secrets are kept out of logs and error messages, and the vault locks on command or after a period of inactivity.

We chose local-only storage to avoid network attack surface and to keep the project scope manageable. We chose per-file keys over one shared key to limit the effect of any single failure. The application stores card details for reference and does not process payments. It protects data at rest, but it does not defend against malware that can read process memory while the vault is unlocked.

A new feature, such as a notes store, can be added by writing one more feature module that uses the Key Manager and Secure File Store. The Menu Controller can later be replaced by a

graphical interface without changing any other layer.




User
selects and acts
Presentation Layer
Menu Controller
[ui.cpp]
unlocks and locks
opens features
Security Layer
Feature Layer
Password Manager
[passwords.cpp]
Authenticator
[totp.cpp]
Vault Manager
[vault.cpp]
Bank Card Wallet
[wallet.cpp]
Backup Service
[backup.cpp]
derives keys
gets file keys
reads salt
reads and writes files
Storage Layer
Key Manager
[keys.cpp]
Secure File Store
[storage.cpp]
encrypts and decrypts
saves to
Crypto Service
[crypto.cpp]
Encrypted local files
vault.meta, passwords.enc,
authenticator.enc, cards.enc,
backup.enc

Digital Wallet sequence diagram

Unlock the vault, then add a password entry

